

Unde lucrezi: `vanilla-ts/src/exercitii/`, peste ce ai făcut la fișa de moștenire. Exercițiile 5 și 7 cer un folder nou în `vanilla-ts/src/fise/`, în care copiezi clasele de care ai nevoie (copiezi, nu imporți: fiecare `main.ts` rămâne de sine stătător).

Nou, ai voie: tipul de bază ca tip al variabilei, al array-ului sau al parametrului; `override`; `super`.

Interzis pe toată fișa (în afara exercițiului 7, unde e scopul): `instanceof`, `typeof`, `if / switch` după numele sau tipul obiectului, `any`.

Cum se citește fișa: **Teorie** înainte de exercițiu, **Indiciu** după prima încercare, **leșirea așteptată** literal. Unde scrie `git diff`, chiar rulezi comanda și te uiți ce linii apar.

Partea 1 — intră în listă fără să atingi bucla

1. O verificare nouă

nivel 1

Fișierul `05-verificari/main.ts`. Scrie:

- `class IncepeCu` extends `Verificare`, câmp `readonly prefix: string`, `constructor(prefix: string)` cu `super("IncepeCu")` pe prima linie.
- `trece(valoare: string): boolean` întoarce `true` dacă valoarea începe cu prefixul.
- Adaugă `new IncepeCu("ESID-")` ca **ultimul** element al array-ului `verificari`. Bucla de afișare rămâne neatinsă.

leșire așteptată:

```
"ESID-100" NuEGoala: OK AreLungimea: OK EDinLista: FAIL IncepeCu: OK
"Active" NuEGoala: OK AreLungimea: FAIL EDinLista: OK IncepeCu: FAIL
" " NuEGoala: FAIL AreLungimea: FAIL EDinLista: FAIL IncepeCu: FAIL
```

Apoi rulează `git diff src/exercitii/05-verificari/main.ts`.

Teorie: `C1-polimorfism-override` — `ColoanaData` și `ColoanaSuma` intră în `coloane: Coloana[]`, iar bucla cheamă `afiseaza` fără să știe pe cine.

Indiciu: `valoare.startsWith(this.prefix)`. Clasa arată exact ca `AreLungimea`, cu un `string` în loc de `number`.

Gata când: ies cele trei linii de mai sus, iar în `git diff` apar doar linii cu `+`: clasa nouă și linia din array. Bucla nu apare.

De reținut: bucla n-a aflat că există un tip nou. Asta e polimorfismul: codul care folosește nu știe cine îl folosește.

2. Markdown la export

nivel 1

Fișierul `06-exportatoare/main.ts`. Scrie:

- `class ExportatorMarkdown` extends `Exportator` (direct din `Exportator`, nu din clasa cu separator).
- `readonly extensie = "md"`.
- `converteste(randuri: string[][]): string`: fiecare rând devine `| valoare1 | valoare2 |`, rândurile pe linii separate.
- Pune `new ExportatorMarkdown()` la finalul array-ului.

leșire așteptată (după blocul `raport.tsv`):

```
raport.md
| ESID-100 | Active |
| ESID-777 | Pending |
```

Teorie: `C1` ; forma clasei e cea a lui `ExportatorJson` din 06.

Indiciu: pentru un rând: ``| ${rand.join(" | ")} |``. Apoi `map` peste rânduri și `join("\n")`, ca la `Csv`.

Gata când: apar cele trei linii, iar bucla de export nu apare în `git diff`.

De reținut: ai adăugat un format de fișier fără să atingi codul care exportă.

3. Funcția care primește baza

nivel 1

Fișierul `05-verificari/main.ts`. Scrie o funcție (în afara oricărei clase):

```
function careNuTrec(verificari: Verificare[], valoare: string): string[]
```

Întoarce numele verificărilor care **nu** trec pentru valoarea dată, în ordinea din array. În funcție nu apare niciun nume de verificare și niciun `if` după nume.

În `main`:

```
console.log(careNuTrec(verificari, " ").join(", "));
console.log(careNuTrec(verificari, "ESID-100").join(", "));
```

leșire așteptată:

```
NuEGoala, AreLungimea, EDinLista, IncepeCu
EDinLista
```

Teorie: `C1` — parametrul `coloane: Coloana[]` primește orice subclasă.

Indiciu: două apeluri înlănțuite: `verificari.filter(v => !v.trece(valoare)).map(v => v.nume)`.

Gata când: ies cele două linii; funcția are cel mult trei linii.

De reținut: parametrul de tip `Verificare` acceptă orice subclasă, și pe cele scrise azi, și pe cele de mâine.

Partea 2 — polimorfismul face muncă

4. Raportul din spatele variabilei

mai greu

Fișierul `03-raport/main.ts`. În `main` adaugă exact codul:

```
const rapoarte: Raport[] = [
  new Raport("Simplu"),
  new RaportSemnat("Semnat", "andreea"),
  new RaportCuData("Cu data", "2026-10-02"),
];
for (const r of rapoarte) {
  r.adauga("o linie");
  console.log(r.text());
  console.log("===");
}

const r: Raport = new RaportSemnat("T", "andreea");
```

Înainte să rulezi, scrie în răspuns ce crezi că afișează `console.log(r.text()) : T` singur, sau `T` urmat de `--` andreea ? Abia apoi adaugă linia și rulează.

leșire așteptată pentru buclă:

```
Simplu
o linie
===
Semnat
o linie
-- andreea
===
Generat: 2026-10-02
Cu data
o linie
===
```

Teorie: `C1` — ultima buclă: elementele sunt declarate `Coloana`, dar fiecare răspunde ca clasa lui.

Indiciu: tipul scris după două puncte (`:` `Raport`) spune doar ce metode ai voie să chemi. Care `text()` rulează decide obiectul creat cu `new`. Dacă predicția ta a fost „`T` singur”, ai confundat tipul variabilei cu tipul obiectului.

Gata când: predicția și rezultatul sunt scrise amândouă în răspuns, iar bucla afișează cele 11 linii de mai sus.

De reținut: metoda se alege după obiectul construit cu `new`, nu după tipul declarat al variabilei.

5. Validarea unui set de date

mai greu

Folder nou: `src/fise/validare-set/main.ts`. Copiază acolo clasa `SetDeDate` (de la 01, doar baza) și `Verificare` cu toate subclasele (de la 05, inclusiv `IncepeCu`). Fără codul lor de rulare.

Scrie funcția:

```
function valideaza(
    set: SetDeDate, reguli: Map<string, Verificare[]>): string[]
```

1. Pentru fiecare pereche `[coloana, verificari]` din `reguli`, în ordinea din `Map`.
2. la `set.valorile(coloana)` și parcurge-le cu indexul `i`.
3. Pentru fiecare verificare din `verificari` care nu trece, adaugă în rezultat mesajul ``rand ${i}, ${coloana}: ${v.num} FAIL``.

În `main`:

```
const set = new SetDeDate(["ESID", "STATUS"]);
set.adauga(["ESID-100", "Active"]);
set.adauga(["ESID-777", "pending"]);
set.adauga(["ESID-78", "Active"]);

const stari = ["Active", "Pending", "Blocked", "Archived"];
const reguli = new Map<string, Verificare[]>([
    ["ESID", [new AreLungimea(8), new IncepeCu("ESID-")]],
    ["STATUS", [new NuEGoala(), new EDinLista(stari)]],
]);

for (const mesaj of valideaza(set, reguli)) {
    console.log(mesaj);
}
```

leșire așteptată:

```
rand 2, ESID: AreLungimea FAIL  
rand 1, STATUS: EDinLista FAIL
```

Proba: adaugă `new AreLungimea(6)` la finalul listei pentru `STATUS`, fără să atingi funcția. Apare o a treia linie:
`rand 1, STATUS: AreLungimea FAIL`.

Teorie: C1 + exercițiul 3 de mai sus (aceeași idee, cu o buclă în plus).

Indiciu: trei bucle una în alta: `for (const [coloana, verificari] of reguli)`, apoi `valori.forEach((valoare, i) => ...)`, apoi `for (const v of verificari)`. Rezultatul e un `string[]` în care faci `push`.

Gata când: ies exact cele două linii, apoi trei după probă; în funcție nu apare niciun nume de verificare.

De reținut: asta e testul de date pe care îl faci la serviciu, în mic: regulile sunt obiecte, iar funcția care le aplică nu știe ce reguli sunt.

Partea 3 — provocări

6. Negația

provocare

Fișierul `05-verificari/main.ts`. Scrie:

- `class Negata extends Verificare`, câmp `readonly interior: Verificare`.
- `constructor(interior: Verificare)`: numele e `"Nu"` + numele celei dinăuntru, deci `super("Nu" + interior.nume)`.
- `trece(valoare)` întoarce opusul lui `interior.trece(valoare)`.
- Pune în array, la final: `new Negata(new EDinLista(["Active", "Pending", "Blocked", "Archived"]))`.

leșire așteptată: fiecare linie capătă la final `NuEDinLista: ...`, cu OK/FAIL inversate față de coloana `EDinLista`:

```
"ESID-100" ... EDinLista: FAIL IncepeCu: OK NuEDinLista: OK  
"Active" ... EDinLista: OK IncepeCu: FAIL NuEDinLista: FAIL  
" " ... EDinLista: FAIL IncepeCu: FAIL NuEDinLista: OK
```

Proba: `new Negata(new Negata(new NuEGoala()))` dă aceleași rezultate ca `NuEGoala`, sub numele `NuNuNuEGoala`.

Teorie: C1; în plus, fișa de moștenire ex. 6 (un obiect care ține un alt obiect de tipul bazei).

Indiciu: `Negata` nu știe ce verificare are înăuntru și nici nu trebuie: cheamă `this.interior.trece(valoare)` și pune `!` în față. În clasa ei nu apare numele niciunei alte clase.

Gata când: coloana nouă e inversul exact al lui `EDinLista` pe toate trei liniile, iar proba cu dublă negație trece.

De reținut: o verificare care conține o verificare. Polimorfismul merge și în interiorul obiectului, nu doar în bucla din `main`.

7. Varianta fără polimorfism

provocare

Folder nou: `src/fise/fara-polimorfism/main.ts`. Copiază `SetDeDate` și rezolvă din nou exercițiul 5, dar

fără subclase:

1. O singură clasă `Regula` cu câmpurile `readonly tip: string` (una din `"nuEGoala"`, `"lungime"`, `"dinLista"`), `readonly numar: number`, `readonly lista: string[]`; constructorul le primește pe toate trei.
2. `function trece(regula: Regula, valoare: string): boolean` cu `if (regula.tip === "nuEGoala") ... else if ...`. Aici `if`-ul după `tip` e permis: asta e scopul exercițiului.
3. `valideaza(set, reguli: Map<string, Regula[]>)` ca la 5, chemând `trece(regula, valoare)`; mesajul folosește `regula.tip` în loc de nume.

Apoi adaugă regula „începe cu” în **ambele** variante (fișierul de la 5 și cel de aici) și rulează în fiecare `git diff --stat`. Scrie în răspuns cele două numere de linii schimbate și trei rânduri: unde s-a dus `if`-ul în varianta polimorfică?

Ieșire așteptată: aceleași două mesaje ca la 5, cu `lungime` în loc de `AreLungimea` și `dinLista` în loc de `EDinLista`.

Teorie: C1 citit invers: ce ar fi fost `afiseaza` dacă ar fi existat o singură clasă `Coloana` cu un câmp `tip`.

Indiciu: în varianta cu `if`, regula nouă cere o ramură în `trece` și un nou câmp în `Regula` (prefixul) și o valoare nouă pentru `tip`. În varianta polimorfică cere o clasă nouă și atât. Numărul de linii din `git diff --stat` trebuie să arate asta.

Gata când: ambele variante dau aceeași ieșire, iar cele două numere sunt scrise în răspuns.

De reținut: polimorfismul nu elimină decizia. O mută din funcția care folosește în obiectul care știe, lângă datele lui.